

1. What are the most challenging aspects of the coursework task?

The most challenging part of this coursework was the implementation of the encryption logic correctly. It was quite simple to handle small shift values, as it just required the addition of the shift value to the index of every letter. However, larger shift values proved more complicated, as it required considering the alphabet as one loop that continuously repeats itself. Understanding and applying this concept was essential to ensure the encryption worked correctly for all values.

Another difficult area was processing user input. While the use of while True loops to repeatedly request input from the user prevented errors resulting from invalid entry, designing those loops so that this did not result in poor behavior by the program required careful thought.

2. How did you go about completing the task?

In order to solve the task, I followed step-by-step the documentation provided by the university. In these latter, it was well explained which functions were expected, for what, and how the program should behave. Each function has been implemented following the given specifications.

In doing so, with every advance in the program, I had to individually test that part of it to make sure that it indeed did what it was supposed to before continuing to the next step. In this way, defects in my code were detected at an early stage in handling user inputs or applying the Caesar Cipher logic. Through regular testing of the program, therefore, and by following the documentation, I have successfully completed the assignment accurately and made sure it has met all the requirements

3. What have you learned over the course of completing this coursework task?

The completion of this coursework task gave me an excellent understanding of the technique known as the Caesar Cipher and how it is used in the processes of message encryption and decryption. I also got to know why encryption and decryption are very crucial, mainly for information protection and the security of data.

I also learned how to apply infinite loops-while True-and break statements in real, practical situations within programming. Such a combination helped me handle user input very efficiently by asking for input from the user constantly until valid data is provided.

Also, the understanding of module programming, or as it is also called-functional programming, was developed. Such an approach means that the big program is then divided into smaller functions, and because of it, the code became more readable, more understandable, and easier to maintain for other programmers.